



თბილისის თავისუფალი უნივერსიტეტი

კომპიუტერული მეცნიერებების, მათემატიკისა და ინჟინერიის
სკოლა (MACS[E])

ელექტრო და კომპიუტერული ინჟინერიის პროგრამა

ბეჟანიშვილი ნიკოლოზ
ცხვედაძე ქეთევანი

ელექტრო და კომპიუტერული ინჟინერიის ლაბორატორია - პროექტის ანგარიში
ფინალური პროექტი - Arcade

თბილისი

2026

Annotation

This project was completed as the final assignment for the Electrical and Computer Engineering Laboratory course at Tbilisi Free University. Unlike other projects, this project was unguided. From the idea and bill of materials to creation and assembly, every step of the project has been done by us.

The requirements for the project were also set by us in the project proposal and will be taken into consideration when grading. The idea of an arcade machine with a light gun was inspired by Giorgi Velijanashvili's project completed in the previous year, although the focus in our project was shifted towards building an easy-to use machine with correct mapping of the light gun onto the screen, rather than writing a custom operating system. The requirements set for the project were all met, meaning it was a complete success. This report contains the description of the work completed as well as the thought process behind it. More documentation, instructions and files can be found on our github linked in the end of this document.

ანოტაცია

ეს პროექტი შესრულდა კურსის „ელექტრო და კომპიუტერული ინჟინერიის ლაბორატორია“ ფარგლებში. სხვა პროექტებისგან განსხვავებით, ეს პროექტი არ იყო კურირებული, რაც გულისხმობს რომ ნაწილების სიით დაწყებული, საბოლოო ამწყობამდე ყოველი ნაბიჯი გადავდგით დამოუკიდებლად.

ამ პროექტის მოთხოვნები ჩვენივე იყო გაწერილი პროექტის proposal-ში და იქნება გამოყენებული პროექტის შეფასებისას. მისი იდეა შთაგონებული იყო გიორგი ველიჯანაშვილის გასულ წელს შექმნილი პროექტით, თუმცა ჩვენ ყურადღება გავამახვილეთ გამოყენების სიმარტივესა და დამიზნების ალგორითმის მუშაობის სიზუსტეზე და არა ოპერაციული სისტემის შექმნაზე. საბოლოოდ, პროექტისთვის დაწესებული ყველა მოთხოვნა იქნა დაკმაყოფილებული, რაც ნიშნავს რომ პროექტი შესრულდა წარმატებით. ეს ანგარიში შეიცავს განხორციელებული სამუშაოსა და მის უკან ფიქრის პროცესის აღწერას. უფრო დეტალური დოკუმენტაცია, ინსტრუქცია და დამატებითი ფაილები ხელმისაწვდომია ჩვენს GitHub-ზე

Table Of Contents

Annotation ii

Table Of Contents iii

Table of Figures iii

Introduction..... 1

Preperation 2

Architecture..... 3

 Cabinet 3

 Gun..... 7

CAD Design 15

Conclusion 22

Skills and Technologies..... 23

Individual contributions 24

GitHub repository 25

References..... 26

Table of Figures

No table of figures entries found.

Introduction

This project was completed as the final assignment for the Electrical and Computer Engineering Laboratory course at Tbilisi Free University. Unlike other projects, this project was unguided. From the idea and bill of materials to creation and assembly, every step of the project has been done by us.

The initial idea was to create an arcade machine that featured a light gun and allowed for playing simple light-gun games such as duck hunt. We later modified to project and added physical buttons and a lever to allow for playing more arcade games.

The project consists of two semi-independent parts: the cabinet and the gun (The cabinet is a fully functional arcade machine in and of itself, with 1080p display, traditional arcade buttons and lever and an operating system designed specifically for emulating retro arcade games that runs on a Raspberry Pi inside it. The gun is an addition to the cabinet that allows for playing more arcade games, but the cabinet is fully functional independently as well. For this reason, the project report is also split into two parts dedicated to each part of the project.

Preperation

An important part of the project was the preparation phase. It was the first project in which we had to come up with an idea, find available components, make a bill of materials and plan assembly steps in advance all by ourselves. The initial idea was a very simple cabinet with just a display that plugs into Raspberry Pi. Instead of a light gun, we aimed for something that is better described as a turret. It would be fixed on one specific known location and have two axes of rotation (roll and pitch) that would allow the player to aim. With either rotary encoders or high-quality potentiometers we would then determine the angles of rotation and with relatively simple trigonometric calculation determine the position of the player's aim on the screen. This idea assumed that the turret would be fixed on one very specific location known in advance and embedded into the calculations. Not only was it not fun, but also hard to set up: each time the machine is moved the turret would have to be set up again perfectly flat at same exact position. We wanted more flexibility and ease of use, so we came up with buttons on the turret that would allow for manual adjustment of the turret's home coordinates, allowing the user to calibrate their crosshair. Taking this idea further, we began looking for ways to automate the calibration process for best user experience. We eventually came up with attaching infrared LEDs to the cabinet and putting an IR camera into the turret so that when it is set at a position the turret would automatically calculate its relative position by taking the position of the four infrared lights seen by the camera and calculating the transformation that would transform the image of an even, flat rectangle seen from the very front to the trapezoid seen from the turret's position. We initially thought the math behind these operations would be heavy to run on an ESP32 we were planning to put into the turret, but it would've still done the job relatively fast. It turned out this math was way simpler and ran fast enough to allow for continuous calibration. Moreover, it allowed us to calculate the aiming point without the need for measuring angles of rotation and doing trigonometry. We will get into the details of the calculations in later chapters. With that, we decided to switch from a stationary turret to a handheld gun.

Architecture

In this chapter we will discuss the architecture of project, the specific components we used or planned to use and reasons we chose them over alternatives. Just like the other chapters, this one will be split into two parts: the cabinet and the gun.

Cabinet

For the main computational unit of the cabinet, we went with a Raspberry Pi 5, because we had one available at the lab. In fact, a Pi 4 would suit our needs even better because unlike the newer model, it has a headphone jack for audio output. In order to get audio out of the Pi 5 we used a simple USB-3.5mm adapter from a local store. We used a 19-inch 5:4 ASUS monitor as a display because it has a perfect size for both gun- and button-driven games and its proportions are closest we could get to that of screens of classic arcade machines. Its resolution was more than enough for simulating arcade games and going for higher resolution would not only be pointless, but also cause extra heat from the load it would put on the Pi. We stripped it from the plastic enclosure and mounted onto a wooden plank instead. It only featured a VGA port, so we had to use a VGA-to-HDMI as well as HDMI-to-micro-HDMI adapter to connect it to the Pi, but it the connection worked natively.

We wanted to design the cabinet plug-and go so that anyone could use it, even without an engineering background, so we mounted a C14 port at the back of the cabinet and added a simple switch at the front that turns everything on with one click via relays¹. The cabinet power is split into two main lines – the 12V DC and 220V AC. The 220V AC line is coming from a wall socket and is supplied only to the power supply and the display (through a relay). The 12V line coming out of the power supply via a relay supplies the rest of the cabinet (though the Raspberry Pi operates at 5V so a 5V DC-DC converter is essential). With that many connections, proper wire management is essential, so we were very mindful of each connection², keeping the colors intuitive and routing wires on walls to eliminate tangling. We designed 3D-printable wire clips

¹ More info about the power supply can be found on the Cabinet Connections Diagram in the Attachments.

² See [] in the Attachments

that kept wires in place. WAGO connectors were also wonderful when it came to splitting lines. In the end, we reached our goal. All of the connections are hidden away inside the cabinet and all the user has to do is plug it into a wall socket and press a button on its front. However, if someone ever needs to see the inside either for maintenance or out of curiosity, we put the back wall on hinges to allow for easy access and designed custom magnet holders keeping the back ‘door’ shut.

One of the essential components are the IR LEDs. The SEN0158 we used³ camera lacks proper documentation, so we could not know what specific LEDs to use to get the most out of it. When the camera finally arrived, we had to experiment with different LEDs and how the camera sees them. The operating wavelength turned out to be 940nm (standard IR). As explained earlier, the gun works by taking the four IR lights from the edges of the screen and calculating a homotopy that determines how the camera is positioned relative to the cabinet, then an inverse of that homotopy is applied to the center point of the camera to map it to a position on the screen. In order for this math to work, the gun needs to be able to see all four IR lights at all times from an acceptable distance (we wanted 1-3 meters) as well as from the side. We initially tried using IR LEDs from old IR proximity sensors like FC-51, but their beam turned out to be too narrow, the lights were visible from a good distance, but stepping slightly to the right or left would noticeably weaken the visibility of the furthest LED, likewise, LEDs from old TV remotes also had great brightness but too small angle of intensity. We went on to buy some wide-angle LEDs (120° angle of intensity) to fix this, but as the angle of intensity increased, the intensity drop per distance increased drastically (there is a square relationship: the wider the beam, the less concentrated it becomes, hence an LED with twice the angle would reach four times less far) making them unusable from a distance. The best angle of intensity turned out to be 30 – not too narrow to not reach a player standing slightly offset from the cabinet and not too wide to lose intensity too quickly. More specifically, we used 30mW 50mA 30° 5mm LEDs (L-53F3C). We also found that by clustering them in groups of three we could increase the combined intensity and angle of

³ more on that in the Gun Architecture chapter.

intensity of an IR bulb. Each cluster had three LEDs wired in series through a 150Ω resistor and we had four of these clusters wired in parallel. The optimal value of the resistor can be determined as follows:

$$\frac{12V - 3 * 1.2V \text{ (voltage drop on the LEDs)}}{50mA \text{ (max. current of the LEDs)}} = \frac{8.4V}{50mA} = 168\Omega$$

*Note: The power on the resistor would be $(50mA)^2 * 168\Omega = 0.42W$, so it needs to be at least $1/2W$ resistor. In our case, we used $1W$ resistors for good measure.*

We cut an old perfboard into small rectangles and soldered the LEDs and the resistor onto it, then designed a 3D-printable housing for it, allowing us to not only mask the solders and connections, but also precisely control the horizontal tilt of the light beam. If all the LEDs lay flat against a tilted screen, the concentrated beam points upwards and the camera loses the light from some distance, but pointing them 90° straight isn't good either, because a player would typically hold the gun slightly higher than the screen. The tilt that we went for was 80° . In order to connect the clusters to each other in parallel and not damage the wire during assembly, we included a 1mm deep trace around the inner side of the front cover for the wires.

In order to enhance the gaming experience, we wanted to add speakers and LED strips, but eventually dropped the LED strips as their intense light and heat could interfere with the IR LEDs, but we did add four speakers on top of the cabinet as well as two PAM8610 audio amplifiers. We connected one of the amplifiers to two speakers facing straight towards the player and another one to two speakers tilted down. Unfortunately, Raspberry Pi 5 does not have analog output, so the audio out of it had low-volume white noise no matter how we tried to clear it (adding capacitors to cut low/high frequencies, cutting ground loop, changing adapters had little to no effect on the noise). We reasoned the cost of a DAC HAT for the pi would not justify the purpose of getting a slightly cleaner audio output, so we left it as it is (besides, a little white noise adds to the retro arcade atmosphere!). When an audio track of a game is playing, the noise is barely noticeable, so the noise is only an issue when the arcade is idle. We positioned the amplifies in such a way that the knobs of the potentiometers that adjust their volume pop out of the

cabinet's side wall, allowing the user to control the volume of the speakers⁴. For a more comfortable feel and visual appearance, we designed a 3D-printable custom knob cap for the potentiometers.

There are two main design styles for the console (buttons and lever) of an arcade machine: Sanwa and Seimitsu. They are functionally the same, but we liked Sanwa's appearance better. We could've simply connected the buttons and the lever to the GPIO pins of the raspberry pi, but this would make the wiring very messy and could damage the board, so instead we used a Zero-Delay USB Encoder, which is a PCB that connects to the buttons and lever, reads them and sends the data directly via a USB connection. We bought a kit containing buttons, a lever and a USB encoder, but these can also be purchased separately.

Fun fact: one of us (people doing this project) is left-handed, while the other one is right-handed. We had a huge dilemma when deciding on the layout for the console (buttons on the left, lever on the right or vice versa) unable to decide, we ended up making a tight-fit for it instead of a permanent mount with screws, which makes no noticeable difference when playing, but allows the user to take the console out, rotate it 180°, re-map the buttons and use it with the layout flipped. We made the layout horizontally symmetrical, so both variants are perfectly playable.

⁴ See [] in the Attachments.

Gun

We are not the first to build an arcade machine with a light gun, but the designs we found while researching all had the light gun wired to the arcade. We built a light gun that was wireless gun. The principle is that four infrared markers around the screen give the gun a reference of known geometry. A camera in the muzzle measures how that reference appears from where the gun is, and from that the gun computes where it is aimed. The result is transmitted to the cabinet, where a receiver presents it to the Raspberry Pi as a standard USB pointing device, so no software has to be installed on the host.

IR markers → camera → ESP32 (gun) → radio → ESP32-S3 (cabinet) → USB → Raspberry Pi

Four groups of three infrared LEDs, wired three in series per group with the groups in parallel, sit at the midpoints of the screen's edges and are powered from the cabinet. Each group is compact enough that the camera resolves it as a single point at playing distance.

The gun contains a DFRobot SEN0158 infrared camera, an ESP32, a solenoid for recoil, a 2S2P 18650 pack with charging and protection, and a buck converter supplying 3.3 V. The enclosure is a two-part 3D-printed shell generated from a parametric CAD script.

We initially wanted to use the camera inside a Nintendo Wii's remote. It heavily simplifies the image processing we would otherwise have to do at software level. The image from the camera would have to be filtered some algorithm would have to identify parts of the image as blobs, then calculate the coordinates of their centers. The camera inside the Nintendo Wii's remote does that at hardware-level and can send the coordinates of up to four IR points via I2C at 100 frames per second, which was very convenient. We later discovered that a DFRobot SEN0158 infrared camera has the exact same hardware and much safer enclosure, meaning we didn't have de-solder the camera from the main board and come up with a delicate casing to protect its exposed PCB.

There were two things we had to think about: how to power the gun and where to do the calculations. Technically, the bare essential parts of the light gun – the ESP and the camera – could run from 3.3V/5V, so just one 18650 cell would suffice, but we also wanted to add some haptic feedback – a solenoid kick after each shot simulating a recoil. We went for a 25N solenoid that required 6V, so we went

up to 2S or 7.4V nominal⁵. We were afraid the cells would discharge too quickly, so we put four cells in a 2S2P configuration. In hindsight, a 2S configuration would probably last long enough, reducing the gun's weight and size.

As per the computations, the gun transmits over ESP-NOW, Espressif's connectionless protocol, to a second ESP32-S3 inside the cabinet. We chose this over the alternatives deliberately. Bluetooth would have removed the receiver entirely, but transmits on a scheduled interval that appears as cursor latency and jitter. Wi-Fi to the host would have required a program running on the Raspberry Pi to synthesise a virtual input device, which must then survive operating system updates that replace the root filesystem. (განვავრცოთ).

The protocol carries state rather than events. Every packet contains the complete aim position and button state, and is sent both on change and at a fixed 5 ms interval. A lost packet therefore delays the cursor by one interval instead of dropping an input, which would be the failure mode of an event-based protocol. The receiver releases all buttons if it hears nothing for 100 ms, so a gun that loses power mid-shot cannot leave a button latched on the host.

The receiver presents itself as a USB input device with five buttons and absolute position axes. Linux exposes it as a pointing device, and a single configuration rule marks it as a light gun, after which Batocera sets up its emulators for gun input without further intervention. A trigger pull while off-screen is reported as the secondary button, which matches the reload convention used by arcade software.

The gun performs all of the geometry and transmits only a result: a screen coordinate and a button state. The receiver performs no computation, translating packets into USB reports. The host is unaware that the device is wireless. This keeps the latency-critical decision, whether a shot was on-screen, local to the gun, and leaves the receiver generic enough that a second gun needs no change to it beyond an identifier.

mapping algorithm

The SEN0158 detects up to four bright infrared sources and reports their centroids on a grid of 1024 by 768. No image is transferred and no image processing runs on the microcontroller, which is what

⁵ Although, 7.4V is above the solenoid's 6V rating, considering the voltage drop on the MOSFET controlling it and the fact that the rated voltage is usually below the actual voltage a component can use, we found that it could safely and reliably operate at 7.4V or even 8.2V at full charge.

makes the approach viable on an ESP32 at a high update rate. Its field of view is 33 degrees horizontally and 23 degrees vertically.

Positions are handled in two coordinate systems. Camera coordinates are the sensor's own grid, normalised to a 0 to 1 range. Screen coordinates place (0, 0) at the top-left corner of the visible image and (1, 1) at the bottom-right, so a marker sitting on the rim takes a value slightly outside that range. The four marker positions in screen coordinates are known constants, derived from measurements of the panel. Writing the visible image as W wide and H tall, and the marker offsets as m with a subscript for the edge:

$$\begin{aligned} q_{top} &= (0.5, -\frac{m_b}{H}) & q_{bottom} &= (0.5, 1 + \frac{m_b}{H}) \\ q_{left} &= (-\frac{m_s}{W}, 0.5) & q_{right} &= (1 + \frac{m_s}{W}, 0.5) \end{aligned}$$

For this cabinet $W = 376.3$ mm and $H = 301.1$ mm, with measured offsets of 14.33 mm above the image, 19.67 mm below it and 14 mm at either side.

The sensor reports points in no fixed order, so each frame must first establish which point is which marker. Because the markers lie at the midpoints of the screen's edges, each one sits on an axis through the centre of the pattern, and this property survives the perspective projection. Viewed from the centre of the four points, the markers therefore appear at roughly 12, 3, 6 and 9 o'clock regardless of the panel's aspect ratio.

We take the average of the four points and measure the direction from it to each point. The labels are then assigned by choosing the arrangement whose directions best match the four expected ones, minimising the squared angular difference. With only four points there are 24 possible assignments, so testing all of them is cheaper than any cleverer method. The assignment stays correct while the gun is rolled within about 45 degrees of upright, beyond which the expected directions become ambiguous.

Two flat surfaces (the screen and the camera's image sensor) viewed through a perspective projection are related by a transform called a homography, described by a three-by-three matrix. Because the markers all lie in the plane of the screen, the relationship between where a marker appears to the camera and where it sits on the screen is exactly of this kind.

Written out, a camera position (x, y) and its screen position (u, v) are related through a three-by-three matrix H acting on homogeneous coordinates, where s is a scale factor that divides out:

$$s * \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = H * \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

$$H = \begin{pmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & 1 \end{pmatrix}$$

The matrix is defined only up to scale, so fixing the bottom-right entry at 1 leaves eight unknowns. Eliminating s gives each marker two linear equations:

$$x * h_{11} + y * h_{12} + h_{13} - u * x * h_{31} - u * y * h_{32} = u$$

$$x * h_{21} + y * h_{22} + h_{23} - v * x * h_{31} - v * y * h_{32} = v$$

Each marker supplies two equations relating its camera position to its screen position. Four markers therefore give eight equations for eight unknowns, and the transform is determined exactly rather than fitted approximately. The firmware solves that system by Gaussian elimination each frame, which takes well under a millisecond on the ESP32 and is not what limits the update rate.

Because the solution is exact, nothing is assumed about the gun's distance, angle or roll. The same calculation handles a shot taken square to the screen and one taken from an extreme angle.

Applying the transform to a single point in the camera's image, the one the sights line up with, gives the position on the screen that the gun is aimed at. For a camera point (x, y) the division by the common denominator is what makes the transform perspective rather than linear:

$$u = \frac{h_{11} * x + h_{12} * y + h_{13}}{h_{31} * x + h_{32} * y + 1}$$

$$v = \frac{h_{21} * x + h_{22} * y + h_{23}}{h_{31} * x + h_{32} * y + 1}$$

A shot counts as on-screen when the result falls inside the unit square, that is when both u and v lie between 0 and 1.

The point in the camera's image that corresponds to the sight line is called the boresight. If the camera were mounted perfectly parallel to the barrel and centred on it, this would simply be the centre of the image. In practice a mounting error of a degree or two displaces the aim by centimetres at playing distance. Because this error cannot be pre-determined and hard-coded into the calculations, we made a way to measure it by calibrating after assembly.

Calibration runs the same transform backwards. With the operator holding the sights on the centre of the screen, the firmware solves the transform from screen to camera and maps the screen centre into the image

$$b = H^{-1} * \begin{pmatrix} 0.5 \\ 0.5 \\ 1 \end{pmatrix} \text{ divided through by its third component}$$

(0.5, 0.5, 1)^T is the center of the screen written in homogeneous coordinates. The screen centre is the 2D point (0.5, 0.5). the 1 appended makes it a 3-element vector so a 3x3 matrix can multiply it.

H⁻¹ is the transform run the other way: screen to camera, rather than camera to screen. Multiplying gives the position in the camera's image where the screen center appears.

dividing through by its third component gets back to a normal 2D point. The multiplication produces three numbers, say (α, β, γ), and the actual point is (α/γ, β/γ)

In the firmware this is done by solving the transform in the opposite direction from the same four markers, rather than by inverting a matrix. That point is the boresight, and it is written to the ESP32's non-volatile memory, where it survives power cycles and firmware updates. One calibration shot is enough, because the correction is angular rather than positional. It is exact at the distance at which it was performed, and the small parallax introduced by the few centimetres between the camera and the sight line is negligible at normal playing distances.

Near the edges of the screen, markers fall outside the camera's field of view. Rather than give up, the firmware reconstructs the missing ones from the last frame in which all four were seen, on the grounds that the pattern's appearance changes slowly compared with the frame rate. With three markers visible, the firmware works out how the pattern has shifted, rotated, scaled and skewed since the last frame where all four were visible, and applies the same change to the missing marker's last position. With only two visible, that is not enough information, so the model is restricted to a shift, a rotation and a uniform scale. Below two visible markers, tracking is reported as lost and a trigger pull is treated as off-screen, which is also the correct behaviour when the gun is genuinely pointed away from the screen.

The field of view sets a minimum playing distance, since all four markers must be visible at once for tracking to start. Aiming at the centre of the screen, each marker must fall within half the field of view, which gives two conditions on the distance d:

$$d \geq \frac{\frac{W}{2} + m_s}{\tan \frac{\theta_h}{2}} \quad d \geq \frac{\frac{H}{2} + m_b}{\tan \frac{\theta_v}{2}}$$

With $\theta_h = 33$ degrees and $\theta_v = 23$ degrees, and the geometry above:

$$\frac{188.15 + 14}{\tan 16.5^\circ} = 682 \text{ mm} \quad \frac{150.55 + 19.67}{\tan 11.5^\circ} = 837 \text{ mm}$$

The vertical limit dominates because the panel is 5:4 rather than widescreen, giving a minimum distance of about 0.84 m. Aiming into the corners is more demanding still, and that is where the reconstruction described above does its work.

The tracking mathematics is implemented as a library with no dependency on the Arduino framework, which allows it to be compiled and tested on a desktop computer. We wrote a test harness that models a camera with the SEN0158's field of view and resolution, projects the four markers into it from any given position and angle, rounds the results as the sensor would, presents them in a scrambled order, and compares the computed aim point against the true one.

The tests cover the transform solver, including its rejection of impossible input, marker identification under roll, aim accuracy, reconstruction during marker loss, and the removal of a deliberately introduced camera misalignment by calibration.

Using the cabinet's actual screen and marker geometry, sweeping nine aim points across the screen at rolls of minus 30, 0 and plus 30 degrees:

Distance	Visible at centre	Fewest visible at corners	Max error, four visible	Max error, corner sweep
0.8 m	4	2	0.05 mm	4.40 mm
0.9 m	4	2	0.04 mm	3.02 mm
1.0 m	4	2	0.10 mm	1.71 mm
1.2 m	4	2	0.41 mm	1.63 mm
1.5 m	4	4	0.44 mm	0.79 mm
2.0 m	4	4	0.88 mm	1.42 mm
2.5 m	4	4	1.00 mm	1.65 mm

With all four markers visible the error stays at or below 1 mm throughout, and is dominated by the rounding of the sensor's output rather than by the algorithm. The larger figures in the final column are the cost of reconstructing missing markers, and they grow as the distance falls because more markers are lost for longer.

Symptom	Cause	Resolution
Aim moved opposite to the gun horizontally	The camera is mounted rotated in the shell, mirroring one axis	Two configuration settings describe the mounting orientation; the mathematics is unchanged
Tracking stopped permanently, every sensor read failing	A transfer interrupted by an electrical disturbance leaves the sensor holding the I2C data line low, wedging the bus	The driver counts consecutive failures, clocks the bus manually to release the line, and reinitialises the sensor
Gun ran correctly on USB but not at all on its own battery	Feeding 3.3V to ESP32 VN pin does not provide enough voltage for the microcontroller to start due to internal buck converter.	Powering from 3.3V pin solved the issue.
Host saw a development board instead of a light gun	The Arduino core starts USB before the firmware runs when its serial-on-boot option is enabled, so the gun device is never declared	Disabled that option so the firmware declares the device itself

The pattern across these is worth stating: the algorithmic work behaved as simulated, while every failure encountered was electrical, mechanical or an artefact of the toolchain. This is consistent with the mechanical side of the project, where several dimensions taken from datasheets proved wrong and were only caught by printing the part.

To keep the firmware maintainable after assembly, we added an update mode. Holding the trigger through power-on for five seconds causes the gun to start its own Wi-Fi access point and accept a firmware upload, so the shell does not have to be opened. It hosts the network rather than joining one, which also makes it usable on networks that isolate clients from one another.

- The minimum playing distance of about 0.84 m is imposed by the sensor's field of view and cannot be improved in software. A wider-angle sensor, or markers positioned closer together, would relax it.
- The solenoid is presently switched fully on for the duration of each pulse. As the pack sits above the solenoid's rated voltage, the pulse is kept short; driving it with a modulated signal would allow the force to be tuned in firmware and would remove the overdrive.
- Recoil is triggered by the trigger rather than by the game. Arcade emulators can export game events such as shots fired and damage taken, and routing these back through the receiver would let the solenoid fire only when the weapon fires in-game.
- Two-player support requires only a second gun with a distinct identifier and a second receiver configured to accept it; no protocol change is needed.

CAD Design

Another big part of this project was designing and assembling the cabinet and the gun in such way that all parts connect well, the design looks plausible and is easy to maintain, meaning the components are easy to replace/repair. We could have made a simple box and permanently glued all components together, but such build would be hard to maintain in case something breaks. We wanted to make a design that anyone (not just us) could look at, open up if needed and do necessary adjustments easily. For that, we had to design everything from scratch and create a lot of custom parts. As usual, we will discuss the designs of the Cabinet and the gun separately.

We choose plywood as the material of the cabinet body, because it was easily accessible in good quantities, is rigid enough to sustain the heavy display and the impact from pressing the buttons while itself not being too heavy, can relatively easily be cut into any flat shape with a CNC machine we had access too and was not as expensive as, for example, metal. We designed the body of the Cabinet (as well as other custom components) in SolidWorks. Unlike the alternatives like Fusion 360, its native interface allows for easier management of complex designs and its building/rebuilding features simplify the process of trial-and-error a lot.

We went for 8mm plywood and created the design with that measurement in mind, but in hindsight, we should've parametrized the width so that the design could easily be rebuilt for a different width with one simple change. Because wood could only be cut into flat shapes, we made a somewhat rectangular design with the console laid horizontally flat and the screen tilted at 120 degrees so that a standing player could comfortably see the screen. For that reason, we added a hollow 20cm space below the console to elevate it, so that the buttons and the lever could comfortably be reached from a standing position. We had a .SLDPRT files for each of its parts and one .SLDASM file for the body where all the parts are mated. At the top of the body, we added two pointy "horns" which host holes for the speakers and add to the appearance of the cabinet. Other parts also have holes for mounting different components: LED cluster holders and button on the front screen cover, C14 Female port on the back, buttons and lever on the console (front-top) and holes for the potentiometer knobs on the side.

We had two main options for cutting the plywood: a laser and a CNC machine. Using a laser would be easier and more precise, but unfortunately the laser we had access to could only cut through a thin plywood (up to 4mm) with maximal width and height of roughly 20 and 30 centimeters. Assembling a rigid body with such limits would be possible, but unnecessarily complicated (we'd had to divide pieces into parts to fit into the dimensions and then glue them together and cut two of each to make the parts thicker). Using a CNC machine had its disadvantages too: it's not as precise, can move the plywood when cutting and cannot make sharp inner corners⁶. We initially tried making box joints, but for this exact reason it did not work well. Then we had to manually carve the inner corners with a knife to make them sharp, which took a lot of time and effort. Another problem was that the plywood we purchased was not exactly 8mm, but rather a little bit thicker. It could be due to manufacturing flaw or the wood expanding with temperature. Either way, we had to thin the parts down with sandpaper to fit them into 8mm windows. A better approach was binding the parts together with plastic corner braces and screws (Simply connecting two parts with a screw would technically work too, but screwing into a thin 8mm side put the wood at risk, especially because of the layered structure of the plywood, so we wouldn't recommend it). We hang the back wall of the cabinet on hinges so it could easily be opened and closed for maintenance inside the cabinet, just like a door. To keep it shut, we added magnets with 3D-printer holders mounted to the inner part of the side and the inner wall of the back 'door'.

We wanted the components to be as easily replicable as possible, so we deliberately avoided using glue and other ways that implied permanent placement. Instead, we designed 3D-printable holder parts⁷ for most of the components. These holders hold the parts in place with screws, making sure they do not wiggle around while retaining the ability to easily remove them if needed.

⁶ A CNC machine works by driving a rotating spindle across the wood, cutting out the trace with width equal to the diameter of the spindle. This means that inner corners will always be rounded. The spindle simply cannot make a sharp inner corner because of its width.

⁷ There is a dedicated folder with holder parts on this project's GitHub.

SolidWorks' rebuild feature really shined here because changing some parameter in one sketch would cause the entire part to change without any extra effort, and changing measurements was curtail as 3D printer's tolerances, error caused by nozzle width and measurement tools made it hard for a print to work as intended on the first try. We used models from GrabCAD to create some of the holders, their authors are credited in the References.

This project was a great way to gain experience in 3D modeling, operating a 3D printer and a CNC machine, leaning about various parameters for slicing 3D prints and getting proficient in the software required. The workflow was as follows:

Designing a flat part in SolidWorks -> Exporting it as .dxf -> Adding toolpaths in ArtCAM
-> Exporting as [] for CNC

Designing a 3D printable part in SolidWorks -> Exporting it as .3mf -> Slicing in Creality Print -> Exporting as .gcode for 3D Printer.

Gun

The gun had to house a specific set of parts: SEN0158 infrared camera, a solenoid, a battery pack of four 18650 cells, a Type-C charger, a buck converter, an ESP32 microcontroller board, a MOSFET and a trigger mechanism. Rather than adapt an existing model, we designed the shell around those parts, sizing every internal holder from measurements of the components in hand.

The model is not a file of clicked operations but a script. We used CadQuery, a Python library in which geometry is expressed as code, so that every dimension is a named value at the top of the file and the whole shell regenerates when one of them changes. This mattered more than it might appear: correcting a measurement became a one-line edit followed by a regeneration, rather than a hunt through a feature tree for everything that depended on it. The script also allowed checks that would be tedious by hand.

The body is roughly 187 mm long, 145 mm tall and 44 mm wide, split into two halves along the centreline. The halves join with six M3 heat-set inserts and M3×12 screws. No other fasteners are used inside: every component is retained by printed features.

Each component has a holder shaped for it. The solenoid sits in a braced cradle, the microcontroller between side guides with end stops, the charger on rails with hold-down tabs, the buck converter in a rimmed pocket, and the battery pack on ledges with stop tabs. The camera screws into an M18 nut trapped in a hexagonal pocket at the muzzle. The MOSFET stands against the rear wall, held by a printed pin and a snap-on clip. A channel runs over the top of the solenoid, notched through both ribs of its cradle, carrying wires from the rear of the gun to the electronics at the front.

We printed both halves printed in PETG on a Bambu Lab P2S. Each on its flat outside face with the interior upward, an orientation chosen so that the internal features are built without overhangs. Supports are needed only beneath the trigger guard; everything else is shaped to avoid them. We printed the trigger separately and solid, since it takes the operator's finger force on every shot and a sparse infill would be the weakest point in the mechanism.

Setting	Value
Nozzle	0.4 mm
Layer height	0.2 mm
Walls	3 to 4
Infill	15 to 20 per cent
Supports	Trigger guard only

Printing a full half takes several hours, which is too slow a loop for checking whether a pocket is the right size. We therefore exported small sections of the model around each critical fit, each printing in well under an hour, and used them to verify dimensions before committing to a full print. This proved to be the most useful decision in the mechanical work. Every fit that later needed correcting was caught on a test piece rather than on a finished half.

We purchased trigger assembly (a trigger, a microswitch and a spring) before the design began, but its pivot pegs sit in one corner of its mounting plate, which would have forced the microswitch into an awkward position given where the solenoid and battery sit. Instead, we designed the trigger as part of the shell which allowed the pivot, arm, stops and spring anchor to be positioned together, and produced a simpler mechanism than working around the bought one would have. We also could not measure the free angle of a torsion spring's legs in advance, so instead of committing to one anchor position, the design provides four fins covering a range of angles, and the leg is placed in whichever gives the wanted tension. This converts an unknown into an adjustment made at assembly.

Several dimensions taken from datasheets or from estimates proved wrong, and each was found by handling a printed part rather than by inspecting the model. The model can only check the numbers it is given.

Assumed	Measured	Consequence
Solenoid plunger protrudes 15 mm when fired	20 mm	Only 0.6 mm of clearance behind it; the solenoid was moved 4 mm forward
Trigger spring coil is 6.5 mm across	Bore is 6.4 mm	The spring post was too large and would not accept the spring
Roller switch matches the datasheet it is copied from	Sits 3.3 mm taller	The switch would have been permanently pressed
Spring coil clears the solenoid	It does not	The solenoid's own return spring had never been measured or modelled

The pattern is consistent: the failures were in the inputs, not in the geometry produced from them. This is the reason for the test pieces in section 3.1, and the reason the revision history of the model reads as a sequence of corrections rather than a straight line to a finished design.

Late in the design, we searched the interior computationally for routes between every pair of components. This revealed a problem that none of the views had made obvious: the solenoid's cradle walled the rear of the gun off from the front, leaving effectively no room for wires where three had to pass. We added a channel over the solenoid after the first test print of the full casing.

- The design is specific to the components listed. Substituting a different solenoid, charger or board requires editing the dimensions at the top of the script and regenerating, rather than modifying the exported files.
- The shell is printed in PETG, chosen for its toughness relative to PLA. A solenoid firing repeatedly imparts a shock to its cradle on every shot, and long-term fatigue at that mounting has not yet been assessed.
- Assembly requires the wiring to be soldered in place before the halves are closed, so servicing any connection means opening the gun. This is what motivated the wireless firmware update mode described in the following section.

Conclusion

Before starting the project, we wrote a project proposal, in which we set some specific goals to ourselves, and we are happy to say that we overcomplete them! The aiming precision is well within 1-1.5cm when calibrated properly; reaction time is almost instant, well within the 100ms we set to ourselves; the system runs stable without powering off for well over the 30 minutes we set (it hasn't failed or powered off so far despite running for hours); it has over 10 pre-set games (we set a minimum of only 1) and more games can easily be added downloaded from open libraries and to the software; the light gun does detect shots within a 45-degree window.

This project was an amazing opportunity to learn CAD and 3D printing skills, gain proficiency with SolidWorks, Autodesk ArtCAM, integrate AI into CAD design; learn to operate a CNC machine and work with wood (cutting, graining, painting, etc) use modern optimized algorithms to run efficient math for mapping and work with and organize complex assemblies with interconnected parts.

One unwritten goal we set for ourselves was ease of use. We wanted to design something absolutely everyone could come to and figure out in under a minute, even without engineering knowledge or prior instructions. What we ended up creating is a functional arcade machine playable with both a gun and a console and all one needs to use it is to plug it into a wall socket and press a big power button on its front. One of the best experiences has been seeing course mates, friends and strangers use it, have fun with it without bothering themselves with the complex connections and algorithms backing up the system. This is the level of abstraction one gets with production-level hardware and being able to achieve that on our own has been a huge achievement!

Skills and Technologies

- Visual Studio Code (with PlatformIO for ESP programming)
- SolidWorks for CAD design.
- CorelDRAW for editing vector cut designs.
- Autodesk ArtCAM for creating files for CNC.
- Programming/markup languages: C++, Python.
- ESP32 microcontroller programming (C++, Arduino framework, PlatformIO)
- Real-time system design with WebSocket.
- ESPAsyncWebServer and AsyncTCP for WiFi access point and WebSocket server.
- NVS (Non-Volatile Storage) for persistent parameter storage across power cycles.
- FreeRTOS: multi-task firmware design, task priorities, core affinity, vTaskDelayUntil.
- Mutex synchronization for shared state in concurrent firmware
- I2C communication protocol for MPU6050.
- Complementary filter design for IMU sensor fusion
- PID controller design, implementation, and empirical tuning
- Laser Cutting machine competence for cutting out and engraving various designs
- Cable management and labeling, designing maintainable and easily replicable modular systems.
- Physical assembly skills: Soldering, wood cutting & gluing
- Git version control and GitHub repository management with meaningful commit conventions
- Hardware debugging: multimeter use, serial monitor diagnostics, pin conflict resolution

Individual contributions

Nikoloz Bezhanishvili:

- Preparation and component choice
- Body design using SolidWorks
- CNC toolpath generation with Autodesk ArtCAM
- 3D printing and slicing
- Operating a CNC machine
- Physical assembly, component mounting, and cable management
- Hardware soldering and wiring
- Project testing and demonstration preparation
- GitHub repository setup and commit management
- Research
- Project report writing

Ketevani Tskhvedadze:

- Body design using SolidWorks
- Gun 3D model design and testing
- Light gun mapping algorithm utilizing and tuning
- 3D printing (and access to Bambu Lab P2S Printer)
- Physical assembly, component mounting, and cable management
- Hardware soldering and wiring
- Project testing and demonstration preparation
- GitHub repository setup and commit management
- Research
- Project report writing

GitHub repository

This project can basically be re-made from scratch by following our GitHub repository. All CAD files (.SLDASM, .SLTPRT), 3D printing files (.3mf), vector files (.dxf). All code and algorithms, as well as connection diagrams for this project are publicly available on GitHub. We've also included a .svg file for laser-cutting, so the body can be cut with a powerful enough laser, giving more precision. The repository is organized into the following subdirectories:

- Cabinet/ — All files related to the cabinet
 - Batocera/
 - CAD/ — The main assembly containing all the parts and subassemblies
 - Body/ — The body sub-assembly and all of its parts
 - Holders/ — Holder parts to keep all components in place
 - Parts/ — Component parts primarily downloaded from GrabCAD
 - CNC/ — .dxf files for CNC cutting
 - Laser/ — .svg files for laser-cutting (plus a .cdr file that allows for quick editing)
 - Printing/ — .3mf files for all holders and other 3D printable parts
 - Receiver-firmware — a PlatformIO project for the ESP-S3 simulating a mouse
- Gun/ — All files related to the light gun
 - CAD/ — .step and .stl files for the gun model as well as test model
 - Firmware/ — a PlatformIO project for the ESP32 inside the gun
 - Scripts/ — a python script generating the 3D model of the gun

Repository: <https://github.com/tskhvedadzee/ECE-Arcade>

Contributors' profiles:

Nikoloz Bezhanishvili – <https://github.com/nbezh>

Ketevani Tskhvedadze - <https://github.com/tskhvedadzee>

References

Datasheets & Component Information:

[1] DFRobot, "SEN0158 IR Positioning Camera Wiki," DFRobot Documentation. [Online]. Available: <https://wiki.dfrobot.com/sen0158/>.

Software & Frameworks:

[2] "Batocera.linux Documentation," Batocera.org. [Online]. Available: <https://batocera.org/>

[3] "CadQuery Documentation," CadQuery. [Online]. Available: <https://cadquery.github.io/>.

Articles & Resources:

[4] "GUN4IR System," GUN4IR. [Online]. Available: <https://www.gun4ir.com/>.

GrabCAD 3D Models:

[5] "Generic 18650 Battery," GrabCAD. [Online]. Available: <https://grabcad.com/library/generic-18650-battery-2>

[6] "Metal LED Illuminated Latching 16mm Power Push Button On Off Switch 12V," GrabCAD. [Online]. Available: <https://grabcad.com/library/metal-led-illuminated-latching-16mm-power-push-button-on-off-switch-12v-1>.

[7] "PAM8610 Class D 12V 2x15W," GrabCAD. [Online]. Available: <https://grabcad.com/library/pam8610-class-d-12v-2x15w-1>.

[8] "5mm LED Red," GrabCAD. [Online]. Available: <https://grabcad.com/library/5mm-led-red-2>.

[9] "C14 Power Socket Male AC 3 Pin 250V 10A," GrabCAD. [Online]. Available: <https://grabcad.com/library/c14-power-socket-male-ac-3-pin-250v-10a-1>.

[10] "Sanwa OBSF-24 Arcade Button," GrabCAD. [Online]. Available: <https://grabcad.com/library/sanwa-obsf-24-arcade-button-1>.

[11] "Sanwa Button OBSF-30," GrabCAD. [Online]. Available: <https://grabcad.com/library/sanwa-button-obsf-30>.

[12] "Sanwa Joystick JLF-TP-8YT-W," GrabCAD. [Online]. Available: <https://grabcad.com/library/sanwa-joystick-jlf-tp-8yt-w>.

[13] "Solenoid JF-1039B 10mm," GrabCAD. [Online]. Available: <https://grabcad.com/library/solenoid-jf-1039b-10mm-1>.